POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 03 -

CRIANDO UM CRUD COM QUARKUS

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON.....4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA?.....14

REFERÊNCIAS 15

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Agora que já conhecemos o Quarkus e aprendemos como trabalhar com as suas extensões, veremoscomo criar o nosso primeiro CRUD.Passaremos por algumas camadas simples no momento da criação dos nossos diretórios e também na configuração de um banco de dados.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula prática, nós criaremos o nosso primeiro CRUD utilizando o Quarkus. Durante a videoaula, configuraremos o ambiente, integraremos um banco de dados utilizando o Hibernate ORM com Panache e implementaremos operações básicas de Create, Read, Update e Delete.

Mas antes de você assistir as videoaulas, seguem todos os links utilizados durante as explicações:

https://github.com/FIAP/JAVA_QUARKUS

https://programadriano.medium.com/dica-r%C3%A1pida-criando-uma-base-postgresql-gr%C3%A1tis-c84f36265994

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Nesta aula, avançaremos os nossos estudos criando nosso primeiro CRUD com o Quarkus. Exploraremos como configurar um projeto Quarkus, como integrar um banco de dados usando Hibernate ORM com Panache e como implementar operações básicas de CRUD (Create, Read, Update e Delete).Durante esse processo, abordaremos conceitos fundamentais, como criação de entidades JPA, configuração de endpointsRESTful e manipulação de dados no banco.

Vamos criar nosso projeto? Para isso, abra um terminal em seu computador e execute o seguinte comando:

```
quarkus create app quarkus-crud --extensions="resteasy-reactive, resteasy-reactive-jackson, hibernate-orm, jdbc-postgresql, hibernate-orm-panache"
```

Esse comando vai criar uma estrutura básica do seu projeto, junto com os pacotes que nós utilizaremos para o nosso CRUD. Para conferir as extensões que foram instaladas, basta executar o seguinte comando em seu terminal:

```
quarkus ext list
```

Caso o comando seja executado com sucesso, o seguinte resultado será exibido em seu terminal:

```
(base) thiagoadriano@Mac quarkus-crud % quarkus ext list
Current Quarkus extensions installed:

* ArtifactId                                    Extension Name
* quarkus-hibernate-orm                         Hibernate ORM
* quarkus-hibernate-orm-panache                 Hibernate ORM with Panache
* quarkus-jdbc-postgresql                       JDBC Driver - PostgreSQL
* quarkus-resteasy-reactive                     RESTEasy Reactive
* quarkus-resteasy-reactive-jackson             RESTEasy Reactive Jackson
```

Figura 1 - Pacote que utilizaremos no CRUD
Fonte: elaborado pelo autor (2024)

Vale lembrar que, aocriar um projeto, nós podemos também adicionar os pacotesposteriormente.

Criando o projeto:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
quarkus create app quarkus-crud
```

Adicionando os pacotes no projeto:

```
quarkus add-extension resteasy-reactive
quarkus add-extension resteasy-reactive-jackson
quarkus add-extension hibernate-orm
quarkus add-extension jdbc-postgresql
quarkus add-extension hibernate-orm-panache
```

Com o projeto criado, vamos configurar os dados de conexão com o nosso banco de dados.

Para isso, abra o arquivo **src/main/resources/application.properties** e atualize-o com o seguinte trecho de código:

```
quarkus.datasource.db-kind=postgresql
quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5432/seu_banco
quarkus.datasource.username=seu_usuario
quarkus.datasource.password=sua_senha
quarkus.hibernate-orm.database.generation=drop-and-create
quarkus.hibernate-orm.log.sql=true
```

Obs.: esses dados estão configurados para um banco de dados local. Porém, como sabemos que nem todos possuem essa configuração, veremos como criar um banco de dados PostgreSQL em uma Cloud grátis.

Para isso, acesse o link supabase.com e crie uma conta. Em seguida, clique em “new project”.

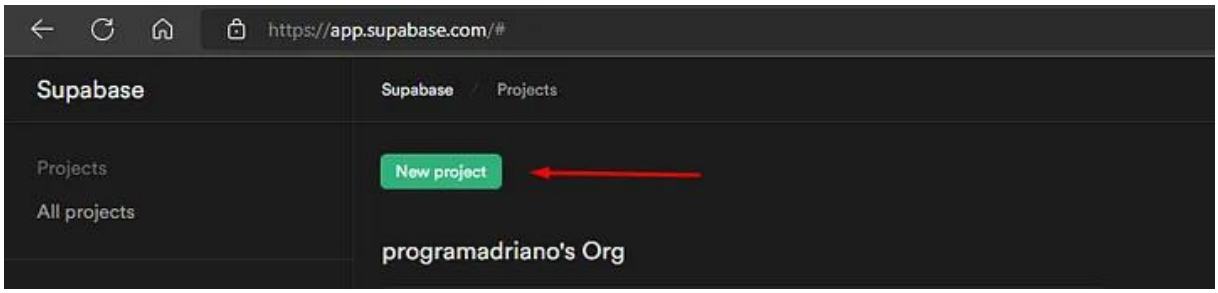


Figura 2 - Criando um banco no Supabase
Fonte: elaborado pelo autor (2024)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Agora, preencha o form com as informações do seu projeto. A seguir, temos uma imagem demonstrando como configuramos para o ambiente:

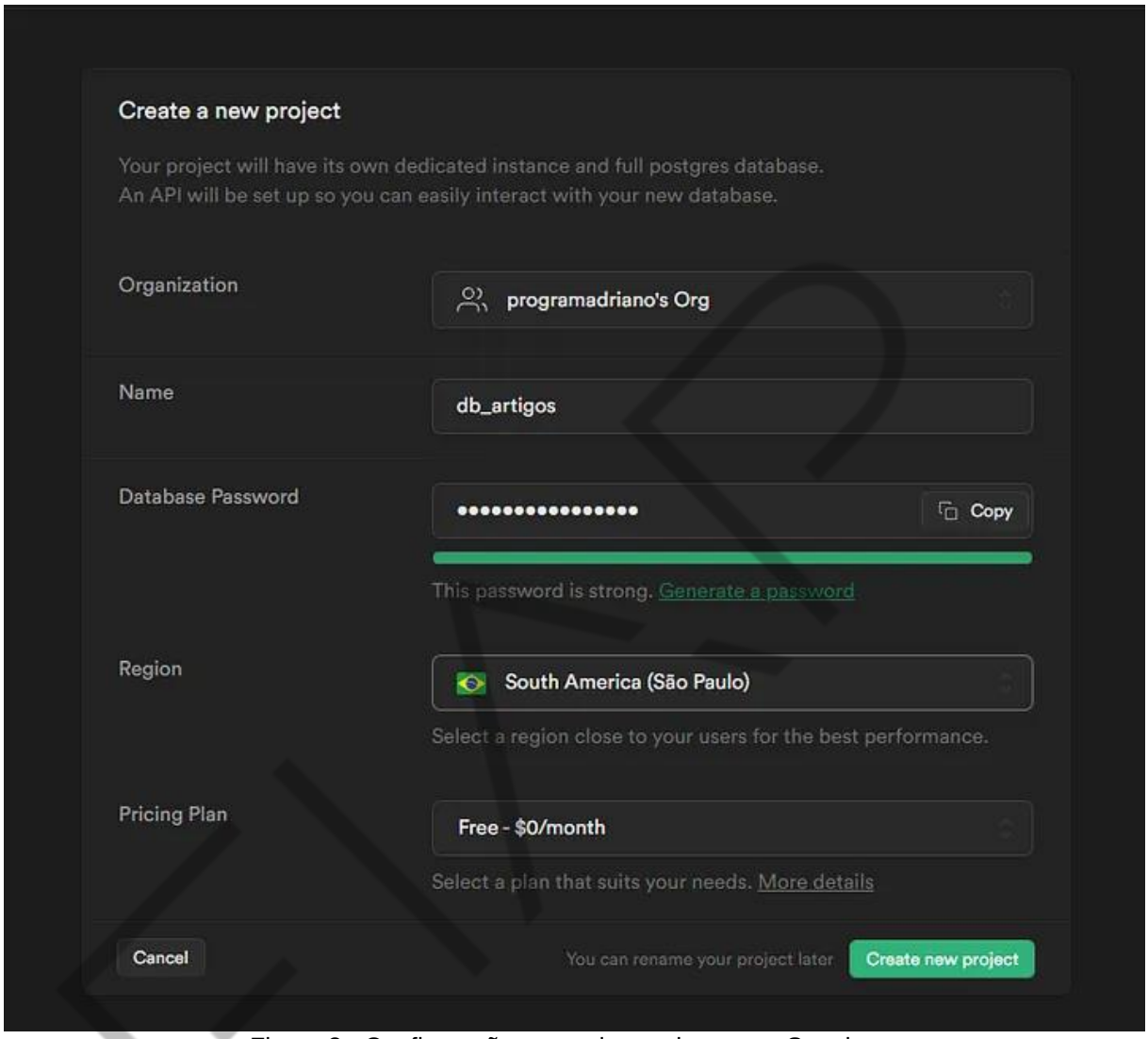


Figura 3 - Configuração para criar um banco no Supabase
Fonte: elaborado pelo autor (2024)

Clique em “**Create new project”** e espere alguns minutos para que o seu projeto seja criado. Em seguida, clique em “Settings ->Database” para copiar os dados de conexão da sua base de dados.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

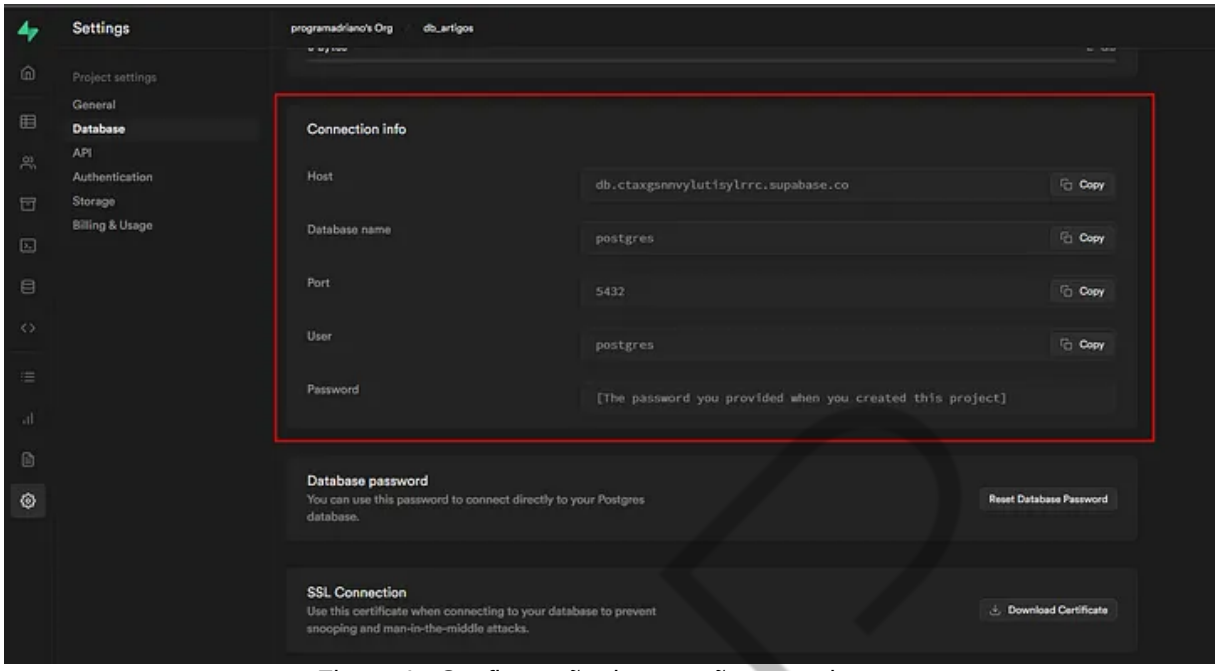


Figura 4 - Configuração de conexão com o banco
Fonte: elaborado pelo autor (2024)

Com o banco criado, vamos criar a nossa entidade. Para o nosso exemplo,criaremos uma entidade chamada “Produto”.

```
packageorg.acme.crud;

importio.quarkus.hibernate.orm.panache.PanacheEntityBase;
importjakarta.persistence.Entity;
importjakarta.persistence.GeneratedValue;
importjakarta.persistence.GenerationType;
importjakarta.persistence.Id;

@Entity
publicclassProdutoextendsPanacheEntityBase {

  @Id
  @GeneratedValue(strategy=GenerationType.IDENTITY)
publicLongid;

publicStringnome;
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
publicDoublepreco;

publicProduto(Stringnome, Doublepreco) {
  this.nome = nome;
this.preco= preco;
  }
}
```

Com as etapas anteriores concluídas, basta criarmos a nossa Resource.

```
packageorg.acme.crud;

importjava.util.List;

importjakarta.transaction.Transactional;
importjakarta.ws.rs.Consumes;
importjakarta.ws.rs.DELETE;
importjakarta.ws.rs.GET;
importjakarta.ws.rs.POST;
importjakarta.ws.rs.PUT;
importjakarta.ws.rs.Path;
importjakarta.ws.rs.PathParam;
importjakarta.ws.rs.Produces;
importjakarta.ws.rs.core.MediaType;
importjakarta.ws.rs.core.Response;

@Path("/produtos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
publicclassProdutoResource {

  @GET
publicList<Produto>listar() {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
returnProduto.listAll();
  }

  @POST
  @Transactional
publicResponsecriar(Produtoproduto) {
if (produto.id!=null) {
returnResponse.status(Response.Status.BAD_REQUEST)
                .entity("ID deve ser nulo ao criar um novo produto")
                .build();
      }
Produto.persist(produto);
returnResponse.ok(produto).status(Response.Status.CREATED).build();
  }

  @PUT
  @Path("{id}")
  @Transactional
publicResponseatualizar(@PathParam("id") Longid, Produtoproduto) {
Produtoentidade=Produto.findById(id);
if (entidade==null) {
returnResponse.status(Response.Status.NOT_FOUND)
                .entity("Produto não encontrado")
                .build();
      }
entidade.nome=produto.nome;
entidade.preco=produto.preco;
returnResponse.ok(entidade).build();
  }

  @DELETE
  @Path("{id}")
  @Transactional
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
publicResponsedeletar(@PathParam("id") Longid) {
if (id==null||id<=0) {
returnResponse.status(Response.Status.BAD_REQUEST)
                .entity("ID inválido")
                .build();
      }

Produtoentidade=Produto.findById(id);
if (entidade==null) {
returnResponse.status(Response.Status.NOT_FOUND)
                .entity("Produto com ID "+id+" não encontrado")
                .build();
      }

try {
entidade.delete(); // Chama o método delete na instância do Produto
returnResponse.noContent().build();
      } catch (Exceptione) {
returnResponse.status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity("Erro ao excluir o produto: "+e.getMessage())
                .build();
      }
   }

}
```

Agora, para que possamos testar o nosso fluxo, vamos criar um arquivo chamado “request.http” e atualizá-lo com o seguinte trecho de código:

Obs.: esse arquivo está configurado com um pacote do Visual Studio Code para funcionar.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
### Listar todos os produtos
GET http://localhost:8080/produtos
Content-Type:application/json

###

### Criar um novo produto
POST http://localhost:8080/produtos
Content-Type:application/json

{
"nome": "Notebook",
"preco": 3500.00
}

###

### Atualizar um produto existente
PUT http://localhost:8080/produtos/1
Content-Type:application/json

{
"nome": "Notebook Gamer",
"preco": 4500.00
}

###

### Deletar um produto
DELETE http://localhost:8080/produtos/1

###
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
### Tentar deletar um produto inexistente
DELETE http://localhost:8080/produtos/999
```

Para testar o nosso código, te convido a assistir nossa videoaula, onde explicaremos projeto com mais detalhes e iremos testá-lo,demonstrando o fluxo todo funcionando.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, nós vimos como criar o nosso primeiro CRUD com Quarkus, passando por algumas camadas simples no momento de criação dos diretórios, e como configurar um banco de dados.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

CRIANDO uma base PostgreSQL grátis: tipagem. **Medium.** 2022. Disponível em: https://programadriano.medium.com/dica-r%C3%A1pida-criando-uma-base-postgresql-gr%C3%A1tis-c84f36265994. Acesso em: 22 jan. 2025.

CRUD. **Mozilla.** [s.d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Glossary/CRUD. Acesso em: 22 jan. 2025.

USING Hibernate ORM and Jakarta Persistence. **Quarkus.** [s.d.]. Disponível em: https://quarkus.io/guides/hibernate-orm. Acesso em: 22 jan. 2025.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Supabase, CRUD, Quarkus.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com